

UT.6.5 PROGRAMACIÓN EN BASES DE DATOS: TRIGGERS

1. INTRODUCCIÓN	2
2. TRIGGERS DE TABLAS	3
3. TRIGGERS DE SUSTITUCIÓN	10
4. ACTIVAR/DESACTIVAR/ELIMINAR TRIGGERS	10

1.INTRODUCCIÓN

En este apartado vamos a tratar una herramienta muy potente e importante proporcionada por PL/SQL para programar nuestra base de datos y mantener la integridad y seguridad que son los **disparadores o triggers**.

Un **trigger** no es más que un procedimiento que se dispara automáticamente cuando se produce un determinado evento en la BD bajo unas circunstancias establecidas a la hora de definirlo.

Los triggers pueden ser de tres tipos:

- **Disparadores de tablas:** Normalmente previenen transacciones erróneas o nos permiten implementar restricciones de integridad o seguridad, o automatizar procesos. Son los más utilizados.
Se ejecutan cuando se realiza alguna sentencia de manipulación de datos sobre una tabla dada. Puede ser lanzado al insertar, al actualizar o al borrar de una tabla, por lo que tendremos disparadores INSERT, UPDATE o DELETE (o mezclados).
Puede ser lanzado una vez por sentencia (por defecto) o una vez por cada fila a la que afecta (FOR EACH ROW). Por lo que tendremos disparadores de sentencia y disparadores de fila.
- **De sustitución:** No se ejecutan ni antes ni después, sino en lugar de (se conocen como triggers INSTEAD OF). Sólo se pueden asociar a vistas y a nivel de fila.
- **De sistema:** Se ejecutan cuando se produce una determinada operación sobre la BD, se crea una tabla, se conecta un usuario, etc. Los eventos que se pueden detectar son por ejemplo: LOGON, LOGOFF, CREATE, DROP, etc. Desde el punto de vista de este curso, estos son posiblemente los triggers menos interesantes, ya que si bien los triggers a nivel de tabla normalmente suelen tener lógica de negocio y por tanto los suele hacer el equipo de desarrollo de la aplicación, los triggers de sistema están asociados a eventos de sistema por lo que lo normal es que la responsabilidad recaiga sobre el equipo de administradores de la BBDD.

Cuando diseñamos un disparador debemos tener en cuenta que:

- No debemos definir disparadores que dupliquen la funcionalidad que ya incorpora Oracle.
- Debemos limitar el tamaño de nuestros disparadores, y si estos son muy grandes codificarlos por medio de subprogramas que sean llamados desde el disparador.
- Cuidar la no creación de disparadores recursivos.

Los TRIGGERS se crean, entre otras razones, por lo siguiente:

- Implementar restricciones complejas de seguridad o integridad.
- Posibilitar la realización de operaciones de manipulación sobre vistas.
- Prevenir transacciones erróneas
- Implementar reglas administrativas complejas
- Generar automáticamente valores derivados.
- Auditar actualizaciones o enviar alertas.

Un disparador puede ser lanzado antes o después de realizar la operación que lo lanza. Por lo que tendremos disparadores BEFORE y disparadores AFTER.

Los permisos necesarios para creación de triggers (CREATE TRIGGER) son específicos (distintos de los de ejecución de SQL por ejemplo).

2. TRIGGERS DE TABLAS

Por lo visto anteriormente, para **definir un trigger** deberemos indicar si será lanzado antes o después de la ejecución de la sentencia que lo lanza, si se lanzará una vez por sentencia o una vez por fila a la que afecta, y si será lanzado al insertar y/o al actualizar y/o al borrar.

La sintaxis que seguiremos para definir un disparador será la siguiente:

```
CREATE [OR REPLACE] TRIGGER nombre_disp
[BEFORE|AFTER]
[DELETE|INSERT|UPDATE {OF columnas}] [ OR [DELETE|INSERT|UPDATE {OF
columnas}]]...
ON tabla
[FOR EACH ROW [WHEN condicion disparo]]
[DECLARE]
-- Declaración de variables locales
BEGIN
-- Instrucciones de ejecución
[EXCEPTION]
-- Instrucciones de excepción
END;
```

Como puedes ver, los tipos de evento que disparan el trigger son las operaciones INSERT, DELETE o UPDATE sobre registros de una tabla.

Pueden dispararse antes o después de la operación: El modificador BEFORE o AFTER indica que el trigger se ejecutará antes o después de ejecutarse la sentencia SQL definida por DELETE, INSERT o UPDATE.

Finalmente, el nivel de los disparadores puede ser la fila o registro o de orden.

- Si incluimos el modificador OF el trigger solo se ejecutará cuando la sentencia SQL afecte a los campos incluidos en la lista.
- El alcance de los disparadores puede ser la fila o de orden. El modificador FOR EACH ROW indica que el trigger se disparará cada vez que se realizan operaciones sobre cada fila de la tabla. Si se acompaña del modificador WHEN, se establece una restricción; el trigger solo actuará, sobre las filas que satisfagan la restricción.

En un trigger de fila, podemos acceder a los valores antiguos y nuevos de la fila afectada por la operación, referenciados a través de las pseudocolumnas :OLD y :NEW. **En este caso el trigger debe estar declarado de forma FOR EACH ROW** Si el disparador es lanzado al insertar, el valor antiguo no tendrá sentido y el valor nuevo será la fila que estamos insertando. Para un disparador lanzado al actualizar el valor antiguo contendrá la fila antes de actualizar y el valor nuevo contendrá la fila que vamos actualizar. Para un disparador lanzado al borrar sólo tendrá sentido el valor antiguo.

Un disparador con nivel de fila se ejecuta una vez por cada fila procesada por la orden que provoca el disparo. Dentro del disparador, puede accederse a la fila que está siendo actualmente procesada utilizando, para ello, dos pseudo-registros, :old y :new. En principio tanto :old como :new son del tipo tabla_disparo%ROWTYPE.

Orden de Disparo	:old	:new
INSERT	No definido; todos los campos toman el valor NULL.	Valores que serán insertados cuando se complete la orden
UPDATE	Valores originales de la fila, antes de la actualización.	Nuevos valores que serán escritos cuando se complete la orden.
DELETE	Valores originales, antes del borrado de la fila.	No definido; todos los campos toman el valor NULL

En el cuerpo de un disparador también podemos acceder a unos predicados que nos dicen qué tipo de operación se está llevando a cabo, que son: INSERTING, UPDATING y DELETING.

En principio, dentro del cuerpo de programa del trigger podemos hacer uso de cualquier orden de consulta o manipulación de la BD, y llamadas a funciones o procedimientos siempre que:

- No se utilicen comandos DDL (creación de tablas, etc,...)
- No se acceda a las tablas que están siendo modificadas con DELETE, INSERT o UPDATE en la misma sesión.
 - Cualquier acceso sobre esta tabla que no sea el que permite el propio trigger (lo veremos más adelante) nos devolverá un error de “Tabla mutando”.
- No se violen las reglas de integridad, es decir no se pueden modificar llaves primarias, ni actualizar claves externas.
- No se utilicen sentencias de control de transacciones (Commit, Rollback o Savepoint).
- No se llamen a procedimientos que transgredan la restricción anterior.

- No se llame a procedimientos que utilicen sentencias de control de transacciones.

Un trigger forma parte de la operación de actualización que lo lanza. Si el trigger falla, falla por tanto toda la operación y Oracle hace Rollback completo.

Si tenemos varios tipos de disparadores sobre una misma tabla, el orden de ejecución será:

- Triggers before de sentencia.
- Triggers before de fila.
- Triggers after de fila.
- Triggers after de sentencia.

Ejemplos:

```
CREATE OR REPLACE TRIGGER EJEMPLO_1
  BEFORE UPDATE OF salemp ON empleado
  FOR EACH ROW
BEGIN
  IF :NEW.salemp <= :OLD.salemp THEN
    RAISE_APPLICATION_ERROR(-20001, 'Intento de bajada salarial
detectado para: ' || :OLD.nomemp);
  END IF;
END;
/
```

- El trigger se ejecutará por cada fila ANTES de una modificación de la columna salemp (salario) de la tabla empleado.
- Dentro del trigger se evalúa una condición y es que su salario no puede ser menor o igual que el anterior. Se lanza una excepción por lo que no llega a realizarse el cambio.

Imaginemos ahora que sí está permitido bajar el salario para los empleados del departamento con código DIRGE :

```
CREATE OR REPLACE TRIGGER EJEMPLO_1
  BEFORE UPDATE OF salemp ON empleado
  FOR EACH ROW WHEN (OLD.coddep != 'DIRGE')
BEGIN
  IF :NEW.salemp <= :OLD.salemp THEN
    RAISE_APPLICATION_ERROR(-20001, 'Intento de bajada salarial
detectado para: ' || :OLD.nomemp);
  END IF;
END;
/
```

Eficiencia (La cláusula WHEN): Al poner la condición `OLD.coddep != 'DIRGE'` en la cabecera, el motor de Oracle es mucho más rápido. Si el empleado es del departamento 'DIRGE', el trigger **ni siquiera llega a dispararse** (no se gasta memoria en abrir el bloque

PL/SQL). **En la cláusula WHEN** los pseudoregistros OLD y NEW se considera una condición de SQL puro y **NO DEBEN** llevar los dos puntos.

¿Cuándo usar BEFORE (El "filtro")?

Se ejecuta **antes** de que los datos se escriban en la tabla. Es el momento ideal para actuar si tu intención es:

- **Validar datos:** Si el dato no cumple una regla (como el ejemplo del salario), lanzas un error y la operación ni siquiera toca la tabla.
- **Modificar valores al vuelo:** Es el único momento donde puedes cambiar un valor de :NEW. Por ejemplo, poner un nombre en mayúsculas antes de guardarlo:
:NEW.nomemp := UPPER(:NEW.nomemp);
- **Asignar IDs automáticos:** Para leer una secuencia y asignarla a la clave primaria antes del INSERT.

Regla de oro: Si necesitas cambiar el valor de :NEW o quieres impedir que se guarde algo incorrecto, usa **BEFORE**.

¿Cuándo usar AFTER (El "notificador")?

Se ejecuta **después** de que la operación de SQL haya tenido éxito (aunque todavía no se haya hecho COMMIT). Se usa cuando:

- **Auditoría y log:** Guardar en otra tabla qué se ha hecho, quién y cuándo. Como el dato ya es "oficial" en la tabla principal, lo registras en la de auditoría.
- **Actualizaciones en cascada:** Si al insertar un empleado quieres actualizar automáticamente el número de empleados en la tabla DEPARTAMENTOS.
- **Acciones que dependen de la fila completa:** Cuando necesitas estar seguro de que la inserción fue correcta antes de lanzar otra acción (como enviar un correo o actualizar una vista materializada).

Característica	BEFORE	AFTER
¿Puede cambiar :NEW?	Sí, directamente.	No, daría error de compilación.
Uso principal	Validación y preparación.	Auditoría y propagación a otras tablas.
Si hay un error...	La fila nunca llega a la tabla.	La fila se "deshace" de la tabla (Rollback).
Rendimiento	Ligeramente más rápido para validaciones.	Mejor para operaciones complejas externas.

Veamos un ejemplo que actualizará el campo NUMHI de la tabla EMPLEADO con el número de hijos que tenga el trabajador en la tabla HIJO. El contador deberá actualizarse al insertar un nuevo registro (sumamos 1), al eliminarlo (restamos 1) o si cambiamos el código del empleado al que pertenece el hijo (sumamos 1 a uno y restamos 1 al otro):

```
CREATE OR REPLACE TRIGGER TR_ACTUALIZA_NUMHI
AFTER INSERT OR DELETE OR UPDATE OF CodEmp ON hijo
FOR EACH ROW
BEGIN
    -- CASO 1: Nuevo hijo registrado
    IF INSERTING THEN
        UPDATE empleado
        SET NumHi = NVL(NumHi, 0) + 1
        WHERE CodEmp = :NEW.CodEmp;

        -- CASO 2: Se borra un registro de hijo. Suponemos datos coherentes
        y no comprobamos negativos
    ELSIF DELETING THEN
        UPDATE empleado
        SET NumHi = NumHi - 1
        WHERE CodEmp = :OLD.CodEmp;

        -- CASO 3: Un hijo cambia de "empleado" (se actualiza su CodEmp)
    ELSIF UPDATING THEN
        -- Restamos al padre que ya no lo tiene
        UPDATE empleado
        SET NumHi = NumHi - 1
        WHERE CodEmp = :OLD.CodEmp;

        -- Sumamos al nuevo padre
        UPDATE empleado
        SET NumHi = NVL(NumHi, 0) + 1
        WHERE CodEmp = :NEW.CodEmp;
    END IF;
END;
/
```

Veamos un ejemplo de auditoría sobre la tabla EMPLEADO. Queremos guardar un log o registro de todas las modificaciones que se realicen en dicha tabla. Para ello vamos a crear previamente la tabla que almacenará el log de movimientos, en la que guardaremos, además de la información modificada, el tipo de operación, el usuario que la realizó y la fecha y hora de la modificación:

```

CREATE TABLE log_empleado (
  CodEmp NUMBER(10),
  CodDep CHAR(5),
  ExTelEmp VARCHAR2(9 CHAR),
  FecInEmp DATE,
  FecNaEmp DATE,
  NifEmp VARCHAR2(9 CHAR),
  NomEmp VARCHAR2(40 CHAR),
  NumHi NUMBER(1) ,
  SalEmp NUMBER(8,2),
  ComEmp NUMBER(6,2),
  TipMov VARCHAR2(10),
  Usuario VARCHAR2(50),
  FechaHora TIMESTAMP
);

```

Ahora generamos el trigger:

```

CREATE OR REPLACE TRIGGER LOG_EMPLEADO
AFTER INSERT OR DELETE OR UPDATE ON EMPLEADO
FOR EACH ROW
BEGIN

IF (INSERTING) THEN

INSERT INTO log_empleado VALUES (:NEW.codemp, :NEW.CodDep,
:NEW.ExTelEmp,
:NEW.FecInEmp, :NEW.FecNaEmp, :NEW.NifEmp, :NEW.NomEmp, :NEW.NumHi,
:NEW.SalEmp, :NEW.ComEmp, 'INSERT', USER, CURRENT_TIMESTAMP);
ELSIF (DELETING) THEN

INSERT INTO log_empleado VALUES (:OLD.codemp, :OLD.CodDep,
:OLD.ExTelEmp,
:OLD.FecInEmp, :OLD.FecNaEmp, :OLD.NifEmp, :OLD.NomEmp, :OLD.NumHi,
:OLD.SalEmp, :OLD.ComEmp, 'DELETE', USER, CURRENT_TIMESTAMP);

ELSE -- UPDATE
INSERT INTO log_empleado VALUES (:OLD.codemp, :OLD.CodDep,
:OLD.ExTelEmp,
:OLD.FecInEmp, :OLD.FecNaEmp, :OLD.NifEmp, :OLD.NomEmp, :OLD.NumHi,
:OLD.SalEmp, :OLD.ComEmp, 'UPDATE-OLD', USER, CURRENT_TIMESTAMP);

```

```

INSERT INTO log_empleado VALUES (:NEW.codemp, :NEW.CodDep,
:NEW.ExTelEmp,
:NEW.FecInEmp, :NEW.FecNaEmp, :NEW.NifEmp, :NEW.NomEmp, :NEW.NumHi,
:NEW.SalEmp, :NEW.ComEmp, 'UPDATE-NEW', USER, CURRENT_TIMESTAMP);

END IF;

END;
/

```

Un disparador de fila no puede acceder a la tabla asociada. Se dice que esa **tabla está mutando**. Si un disparador es lanzado en cascada por otro disparador, éste no podrá acceder a ninguna de las tablas asociadas, y así recursivamente. Se dice que la tabla muta porque Oracle está "en medio" de cambiarla, y no permite hacer consultas sobre ella para evitar inconsistencias (lecturas sucias).

```

CREATE TRIGGER prueba BEFORE UPDATE ON empleado
FOR EACH ROW
BEGIN
...
SELECT nomemp FROM empleado WHERE ...
/*devolvería el error ORA-04091: table EMPLEADO is mutating,
trigger/function may not see it*/...
END;
/

```

Veamos por último un ejemplo de trigger de nivel de sentencia (el que no lleva FOR EACH ROW). Imagina que la política de la empresa prohíbe hacer modificaciones en la tabla empleado fuera del horario de oficina (por ejemplo, fines de semana o antes de las 8:00 am). (Estos triggers son mucho menos comunes).

```

CREATE OR REPLACE TRIGGER TR_SEGURIDAD_HORARIO
  BEFORE INSERT OR UPDATE OR DELETE ON empleado
  -- NOTA: Al omitir "FOR EACH ROW", es un trigger de SENTENCIA
BEGIN
  -- Comprobamos si es sábado ,domingo o si estamos fuera del horario
  de 08:00 a 20:00
  IF TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('SAT',
'SUN')
    OR TO_NUMBER(TO_CHAR(SYSDATE, 'HH24')) NOT BETWEEN 8 AND 19 THEN
    RAISE_APPLICATION_ERROR(-20500, 'Operación no permitida
fuera del horario laboral (Lunes-Viernes, 08:00-20:00).');
  END IF;
END;
/

```

3. TRIGGERS DE SUSTITUCIÓN

Podemos crear triggers que no se ejecutan antes ni después de una instrucción sino en Lugar de (instead of).

Solo podemos utilizar estos triggers si están asociados a vistas, además actúan siempre a nivel de fila. La sintaxis de este tipo de trigger es la siguiente:

```
CREATE [OR REPLACE] TRIGGER nombre_trigger
INSTEAD OF { INSERT | DELETE | UPDATE [OF columnas]}
[ OR { INSERT | DELETE | UPDATE}]
ON nombre vista
[ FOR EACH ROW]
[DECLARE]
declaraciones
BEGIN
instrucciones
[EXCEPTION]
excepciones
END;
/
```

4. ACTIVAR/DESACTIVAR/ELIMINAR TRIGGERS

Para activar o desactivar un trigger se utilizan las siguientes expresiones:

- `alter trigger nombredeltrigger disable;`
- `alter trigger nombredeltrigger enable;`

Para volver a compilar un trigger se utiliza la siguiente expresión:

- `alter trigger nombredeltrigger compile;`

Para eliminar un trigger se utiliza la siguiente expresión:

- `drop trigger nombredeltrigger;`